HW2

October 25, 2026

[13]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

0.1 Question 1

[14]:
```
fg = pd.read_csv('nndb_flat.csv')
fg.head()
```

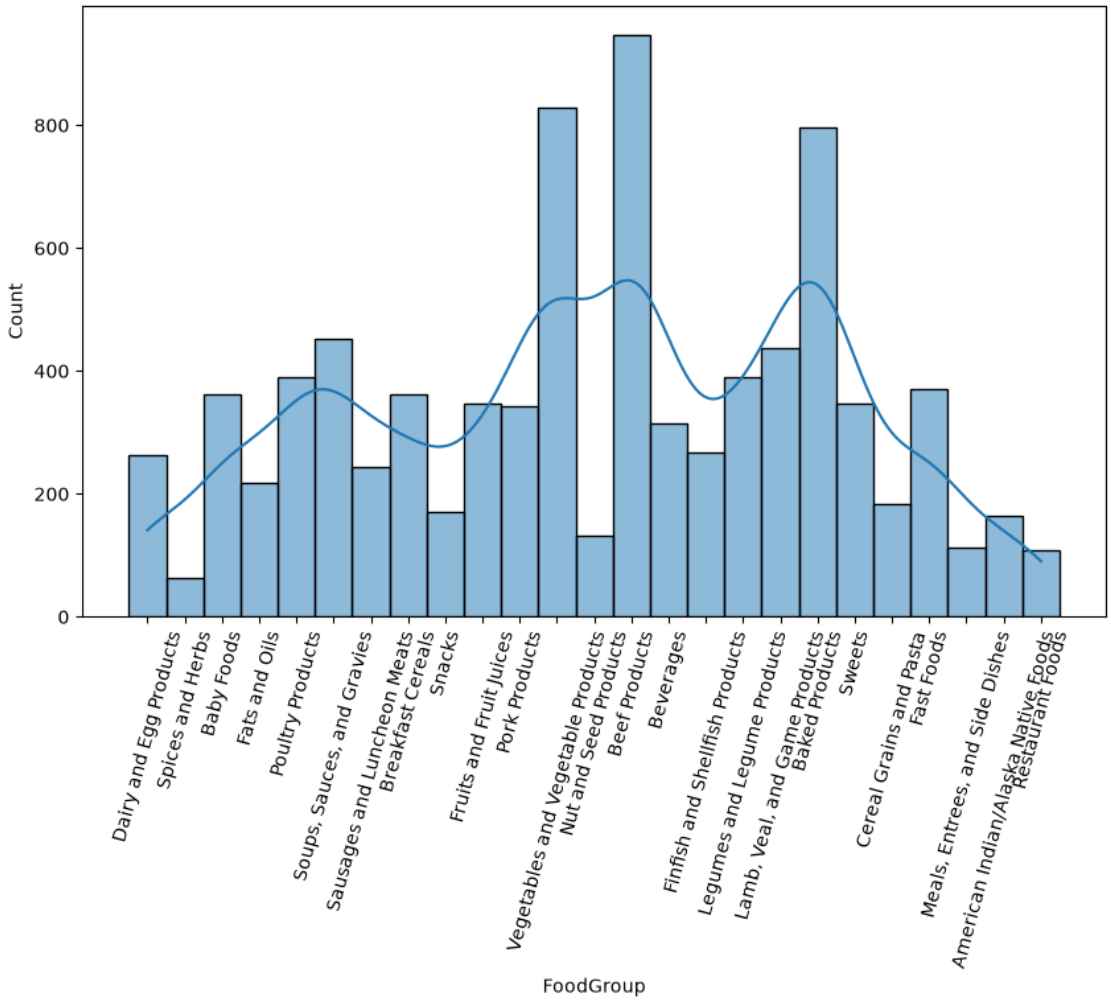
[14]:

	ID	FoodGroup	ShortDescrip	Descrip	CommonName	MfgName	ScientificName	Energy_kcal	Protein_g	Fat_g	…	Folate_USRDA	Niacin_USRDA	Riboflavin_USRDA	Thiamin_USRDA	Calcium_USRDA	Copper_USRDA	Magnesium_USRDA
0	1001	Dairy and Egg Products	BUTTER,WITH SALT	Butter, salted	NaN	NaN	NaN	717.0	0.85	81.11	…	0.0075	0.002625	0.026154	0.004167	0.020000	0.000000	0.004762
1	1002	Dairy and Egg Products	BUTTER,WHIPPED,WITH SALT	Butter, whipped, with salt	NaN	NaN	NaN	717.0	0.85	81.11	…	0.0075	0.002625	0.026154	0.004167	0.020000	0.000018	0.004762
2	1003	Dairy and Egg Products	BUTTER OIL,ANHYDROUS	Butter oil, anhydrous	NaN	NaN	NaN	876.0	0.28	99.48	…	0.0000	0.000188	0.003846	0.000833	0.003333	0.000001	0.000000
3	1004	Dairy and Egg Products	CHEESE,BLUE	Cheese, blue	NaN	NaN	NaN	353.0	21.40	28.74	…	0.0900	0.063500	0.293846	0.024167	0.440000	0.000044	0.054762
4	1005	Dairy and Egg Products	CHEESE,BRICK	Cheese, brick	NaN	NaN	NaN	371.0	23.24	29.68	…	0.0500	0.007375	0.270000	0.011667	0.561667	0.000027	0.057143

	Phosphorus_USRDA	Selenium_USRDA	Zinc_USRDA
0	0.034286	0.018182	0.008182
1	0.032857	0.018182	0.004545
2	0.004286	0.000000	0.000909
3	0.552857	0.263636	0.241818
4	0.644286	0.263636	0.236364

[5 rows x 45 columns]

[15]:
```
plt.figure(figsize=(10, 6))
sns.histplot(data=fg, x='FoodGroup', kde=True)
plt.xticks(rotation=75)
plt.show()
```



I would say that the histogram is overall well balanced with the exception of the categories: Beef Products, Vegetable and Vegetable Products, and Baked Products appearing the most frequently

within the dataset by about 2-3 times more frequently than other categories.

[16]:
```
fg["FoodGroup"].value_counts(ascending = False)
```

[16]:
```
FoodGroup
Beef Products                          946
Vegetables and Vegetable Products      828
Baked Products                         797
Soups, Sauces, and Gravies             452
Lamb, Veal, and Game Products          438
Poultry Products                       390
Legumes and Legume Products            389
Fast Foods                             371
Breakfast Cereals                      363
Baby Foods                             362
Sweets                                 347
Fruits and Fruit Juices                346
Pork Products                          343
Beverages                              315
Finfish and Shellfish Products         267
Dairy and Egg Products                 264
Sausages and Luncheon Meats            244
Fats and Oils                          219
Cereal Grains and Pasta                183
Snacks                                 171
American Indian/Alaska Native Foods    165
Nut and Seed Products                  133
Meals, Entrees, and Side Dishes        113
Restaurant Foods                       108
Spices and Herbs                        64
Name: count, dtype: int64
```

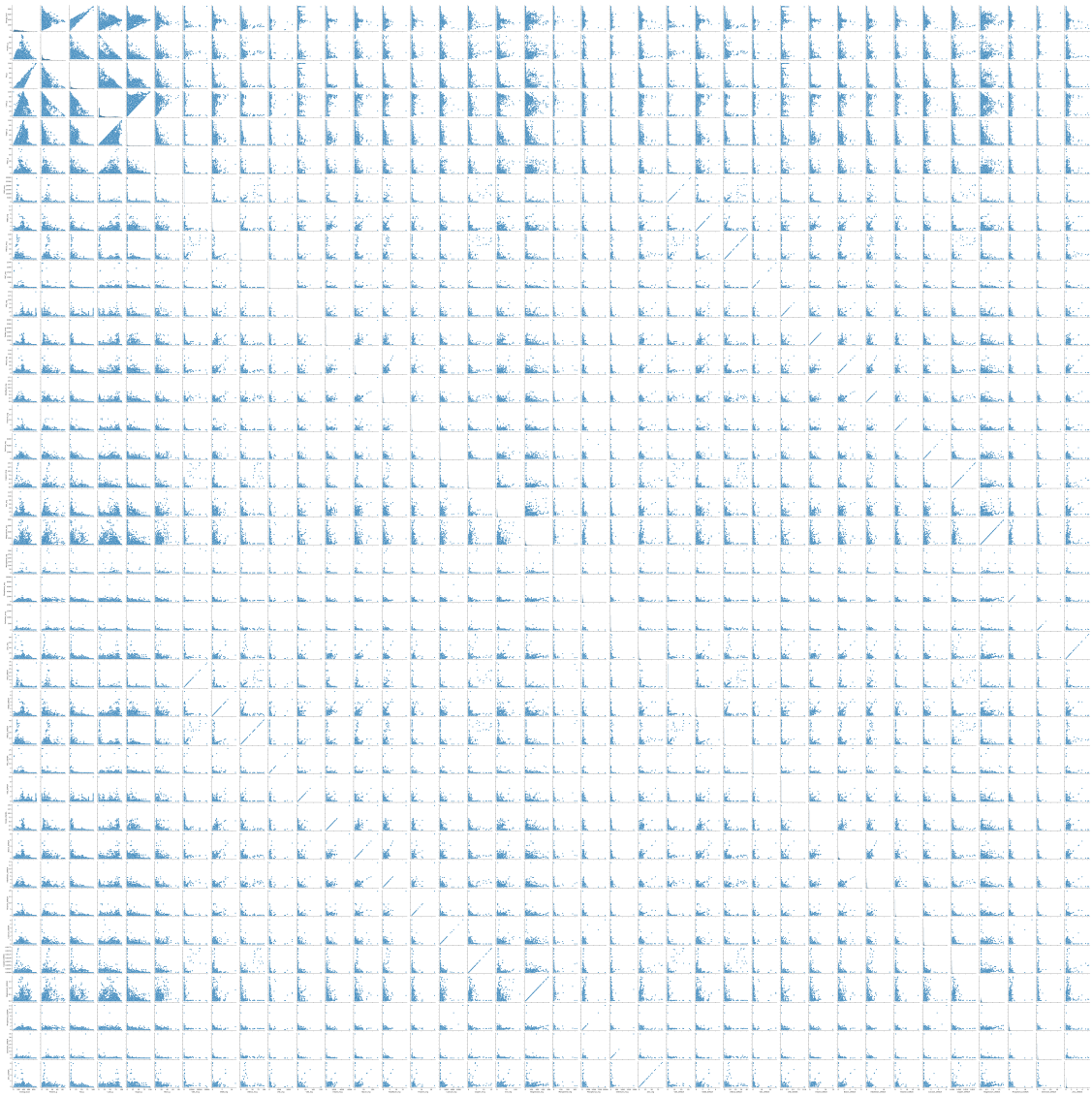
[17]:
```
fg.shape
```

[17]:
```
(8618, 45)
```

[18]:
```
fg_X_cols = fg.drop(columns = ["FoodGroup", "ID", "ShortDescrip", "Descrip",␣
 ↪"CommonName", "MfgName", "ScientificName"])
plt.figure(figsize=(18, 15))
sns.pairplot(fg_X_cols)
```

[18]:
```
<seaborn.axisgrid.PairGrid at 0x29318af5590>

<Figure size 1800x1500 with 0 Axes>
```



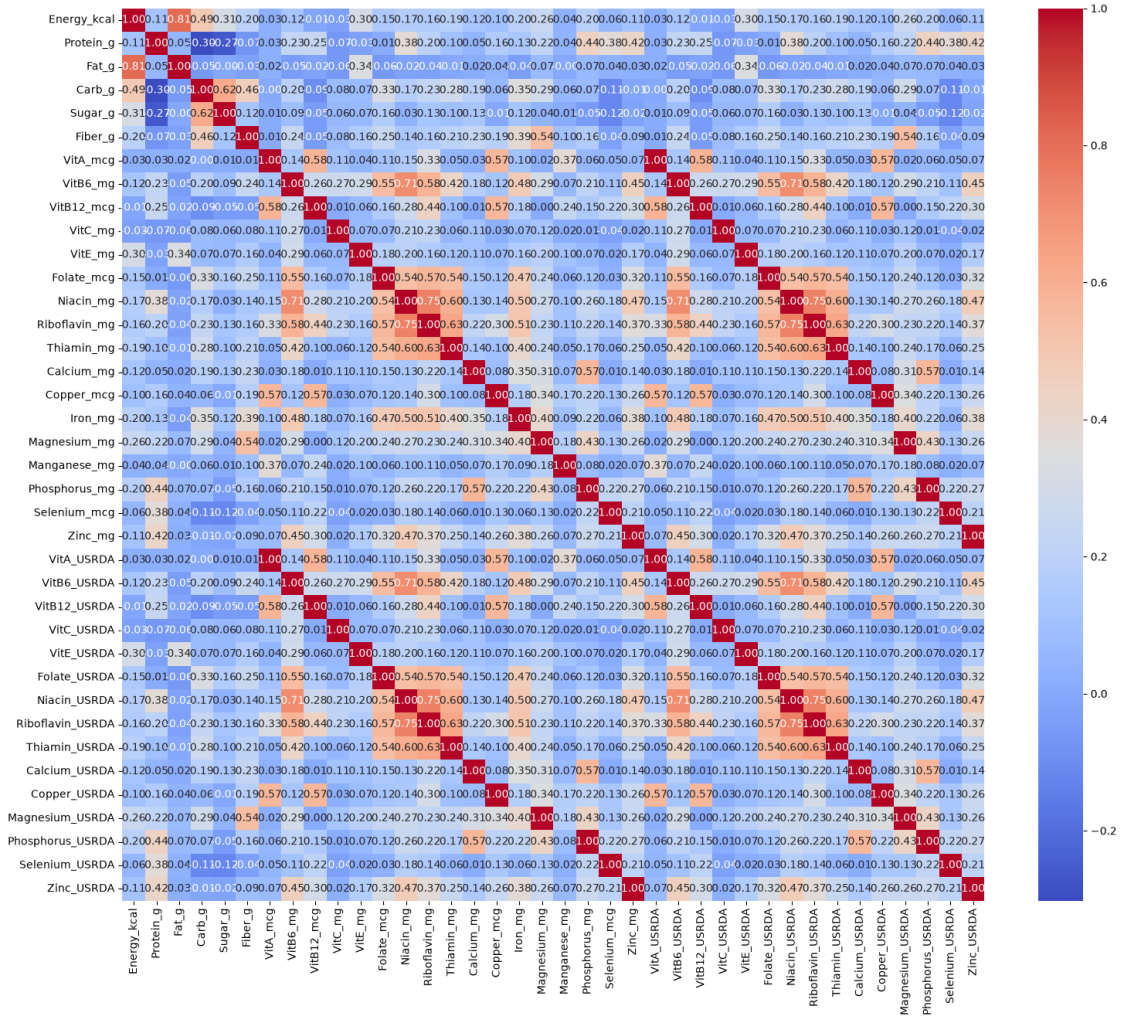
[19]:
```
fg_wo_target.dtypes
```

[19]:
```
ID                 int64
ShortDescrip         str
Descrip              str
CommonName           str
MfgName              str
ScientificName       str
Energy_kcal      float64
Protein_g        float64
Fat_g            float64
Carb_g           float64
Sugar_g          float64
```

```
Fiber_g             float64
VitA_mcg            float64
VitB6_mg            float64
VitB12_mcg          float64
VitC_mg             float64
VitE_mg             float64
Folate_mcg          float64
Niacin_mg           float64
Riboflavin_mg       float64
Thiamin_mg          float64
Calcium_mg          float64
Copper_mcg          float64
Iron_mg             float64
Magnesium_mg        float64
Manganese_mg        float64
Phosphorus_mg       float64
Selenium_mcg        float64
Zinc_mg             float64
VitA_USRDA          float64
VitB6_USRDA         float64
VitB12_USRDA        float64
VitC_USRDA          float64
VitE_USRDA          float64
Folate_USRDA        float64
Niacin_USRDA        float64
Riboflavin_USRDA    float64
Thiamin_USRDA       float64
Calcium_USRDA       float64
Copper_USRDA        float64
Magnesium_USRDA     float64
Phosphorus_USRDA    float64
Selenium_USRDA      float64
Zinc_USRDA          float64
dtype: object
```

[20]:
```
plt.figure(figsize=(18, 15))
sns.heatmap(fg_X_cols.corr(), annot=True, cmap='coolwarm', fmt=".2f")
```

[20]: <Axes: >



The heatmap works way better for this dataset than the pairplot since there are so many possibl

[21]:
```
fg_X_cols.columns
```

[21]:
```
Index(['Energy_kcal', 'Protein_g', 'Fat_g', 'Carb_g', 'Sugar_g', 'Fiber_g',
       'VitA_mcg', 'VitB6_mg', 'VitB12_mcg', 'VitC_mg', 'VitE_mg',
       'Folate_mcg', 'Niacin_mg', 'Riboflavin_mg', 'Thiamin_mg', 'Calcium_mg',
       'Copper_mcg', 'Iron_mg', 'Magnesium_mg', 'Manganese_mg',
       'Phosphorus_mg', 'Selenium_mcg', 'Zinc_mg', 'VitA_USRDA', 'VitB6_USRDA',
       'VitB12_USRDA', 'VitC_USRDA', 'VitE_USRDA', 'Folate_USRDA',
       'Niacin_USRDA', 'Riboflavin_USRDA', 'Thiamin_USRDA', 'Calcium_USRDA',
       'Copper_USRDA', 'Magnesium_USRDA', 'Phosphorus_USRDA', 'Selenium_USRDA',
       'Zinc_USRDA'],
      dtype='str')
```

Based on the correlation heatmap, the USRDA percentage columns sahre a perfect linear combi
'VitA_USRDA', 'VitB6_USRDA',

```
    'VitB12_USRDA', 'VitC_USRDA', 'VitE_USRDA', 'Folate_USRDA',
    'Niacin_USRDA', 'Riboflavin_USRDA', 'Thiamin_USRDA', 'Calcium_USRDA',
    'Copper_USRDA', 'Magnesium_USRDA', 'Phosphorus_USRDA', 'Selenium_USRDA',
    'Zinc_USRDA'
```

[22]:
```
fg_X_cols.drop(columns = ['VitA_USRDA', 'VitB6_USRDA',
       'VitB12_USRDA', 'VitC_USRDA', 'VitE_USRDA', 'Folate_USRDA',
       'Niacin_USRDA', 'Riboflavin_USRDA', 'Thiamin_USRDA', 'Calcium_USRDA',
       'Copper_USRDA', 'Magnesium_USRDA', 'Phosphorus_USRDA', 'Selenium_USRDA',
       'Zinc_USRDA'], inplace = True)
```

```
C:\Users\denni\AppData\Local\Temp\ipykernel_28424\1394264356.py:1:
Pandas4Warning: The inplace keyword in DataFrame.drop is deprecated and will be
removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace
for more details.
  fg_X_cols.drop(columns = ['VitA_USRDA', 'VitB6_USRDA',
```

[ ]:
```
# Looking for missingness
missingness_profile = pd.DataFrame({'Missing_Pct': fg_X_cols.isnull().mean() *␣
 ↪100,'Zero_Pct': (fg_X_cols == 0).mean() * 100})

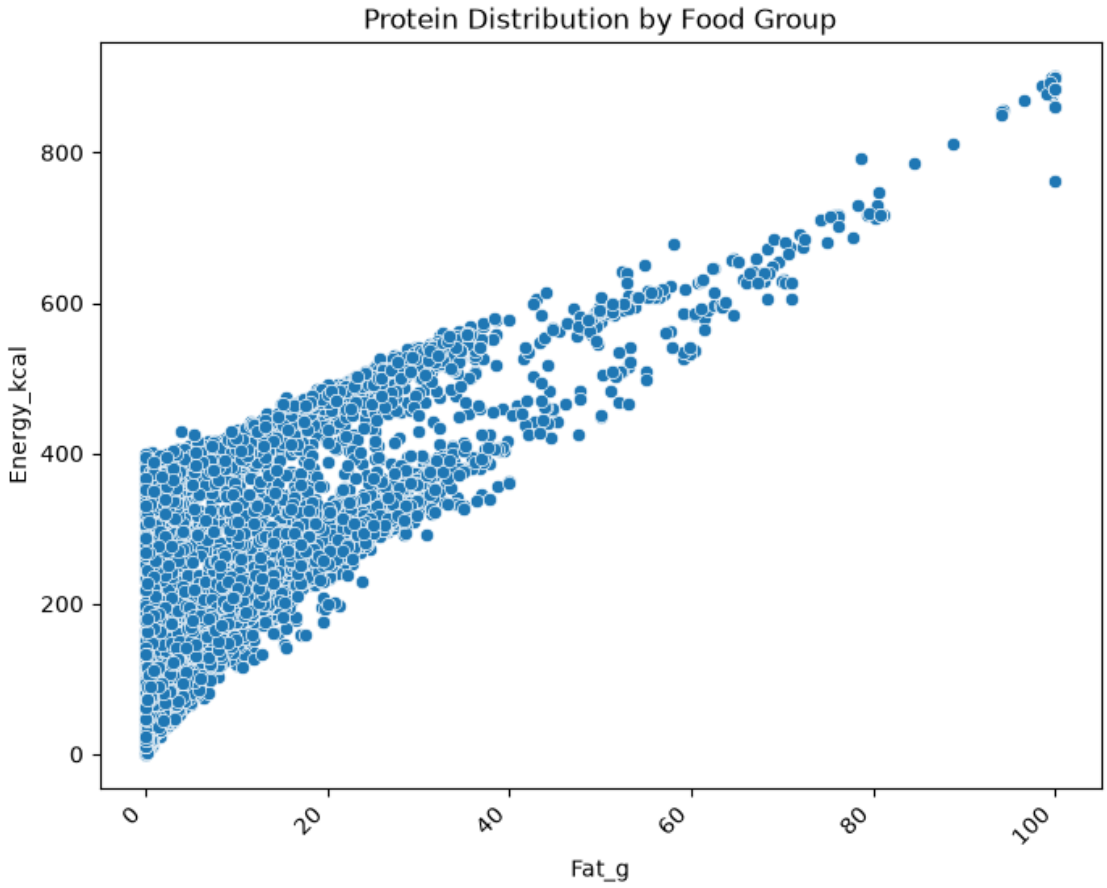
print(missingness_profile.sort_values(by='Zero_Pct', ascending = False))
```

```
               Missing_Pct   Zero_Pct
VitC_mg                0.0  55.848225
Fiber_g                0.0  47.249942
VitB12_mcg             0.0  46.774194
Sugar_g                0.0  46.472499
VitA_mcg               0.0  46.240427
VitE_mg                0.0  39.754003
Folate_mcg             0.0  29.798097
Manganese_mg           0.0  26.583894
Carb_g                 0.0  24.622882
Selenium_mcg           0.0  24.147134
Copper_mcg             0.0  17.045718
VitB6_mg               0.0  14.841030
Thiamin_mg             0.0  11.638431
Niacin_mg              0.0  11.046647
Riboflavin_mg          0.0  11.000232
Magnesium_mg           0.0  10.431655
Zinc_mg                0.0  10.373637
Phosphorus_mg          0.0   9.166860
Calcium_mg             0.0   7.020190
Iron_mg                0.0   4.525412
Fat_g                  0.0   3.794384
Protein_g              0.0   3.747969
Energy_kcal            0.0   0.348109
```

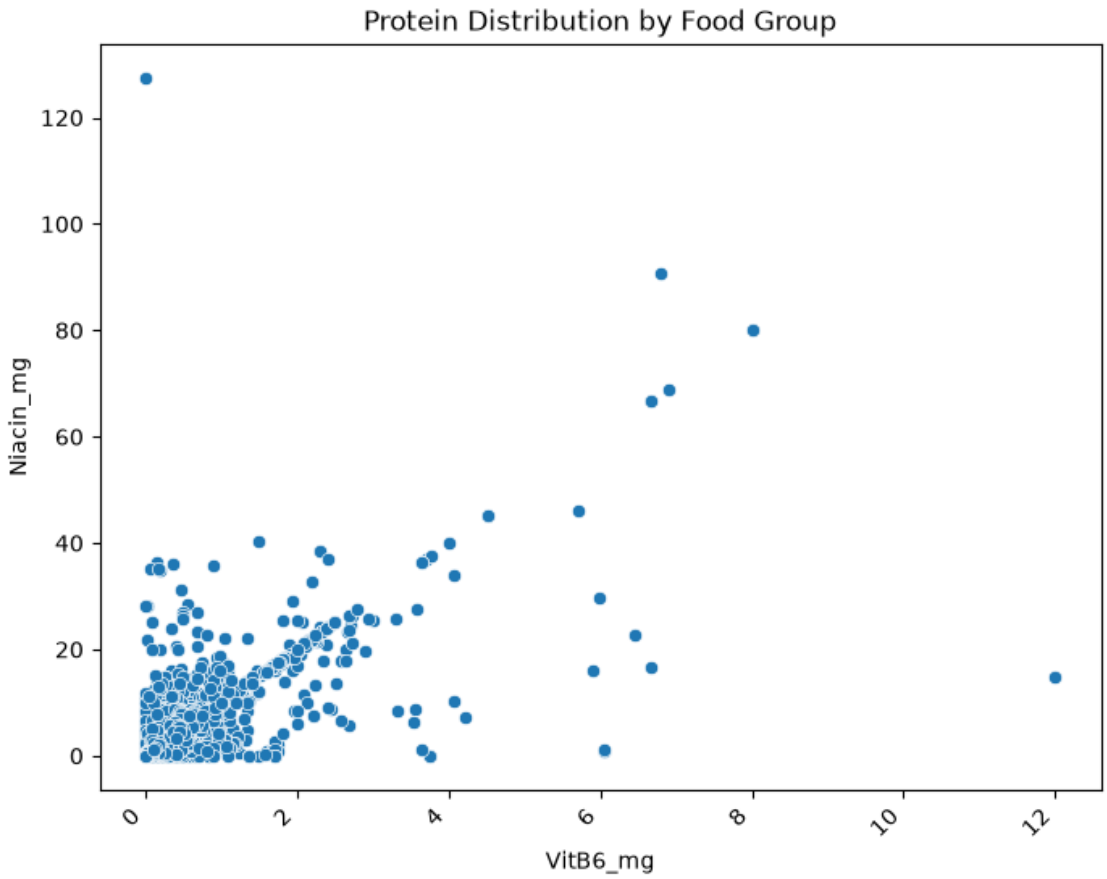
[29]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=fg_X_cols, x='Sugar_g', y='Carb_g')
plt.xticks(rotation=45, ha='right')
plt.title("Protein Distribution by Food Group")
plt.show()
```



[31]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=fg_X_cols, x='Fat_g', y='Energy_kcal')
plt.xticks(rotation=45, ha='right')
plt.title("Protein Distribution by Food Group")
plt.show()
```



[33]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=fg_X_cols, x='VitB6_mg', y='Niacin_mg')
plt.xticks(rotation=45, ha='right')
plt.title("Protein Distribution by Food Group")
plt.show()
```



Candidate X set: 'Protein_g', 'Fat_g', 'Carb_g', 'Sugar_g', 'Fiber_g',
'VitA_mcg', 'VitB12_mcg', 'Niacin_mg', 'VitE_mg',
'Folate_mcg', , 'Thiamin_mg', 'Calcium_mg',
'Copper_mcg', 'Iron_mg', 'Magnesium_mg', 'Manganese_mg',
'Phosphorus_mg', 'Selenium_mcg', 'Zinc_mg'.

I have removed Energy_kcal since it is strong correlated with Fat_g and we have the knowled

The candidate X set can be reduced further, but I believe this is one of the stronger ones

[ ]:
```
X_synthetic = pd.DataFrame()

X_synthetic['Sugar_to_Carb_Ratio'] = np.where(fg_X_cols['Carb_g'] > 0,␣
↪fg_X_cols['Sugar_g'] / fg_X_cols['Carb_g'], 0)

X_synthetic['Total_Macros_g'] = fg_X_cols['Protein_g'] + fg_X_cols['Fat_g'] +␣
↪fg_X_cols['Carb_g']
```

```
X_synthetic['Protein_to_Energy_Ratio'] = np.where(fg_X_cols['Energy_kcal'] > 0,␣
 ↪(fg_X_cols['Protein_g'] * 4) / fg_X_cols['Energy_kcal'], 0)

X_synthetic['Net_Carbs_g'] = np.maximum(0, fg_X_cols['Carb_g'] -␣
 ↪fg_X_cols['Fiber_g'])
```

[38]:
```
X_synthetic.head(10)
```

[38]:

	Sugar_to_Carb_Ratio	Total_Macros_g	Protein_to_Energy_Ratio	Net_Carbs_g
0	1.000000	82.02	0.004742	0.06
1	1.000000	82.02	0.004742	0.06
2	0.000000	99.76	0.001279	0.00
3	0.213675	52.48	0.242493	2.34
4	0.182796	55.71	0.250566	2.79
5	1.000000	48.88	0.248503	0.45
6	1.000000	44.52	0.264000	0.46
7	0.000000	57.44	0.267872	3.06
8	0.210526	59.19	0.236847	1.33
9	0.000000	58.75	0.241550	4.78

Some of the syntethic Xs I created based on the feature engineering work includes a Net Carb fe

0.2 Question 2

[43]:
```
df = pd.read_csv("reg1.csv")
df.head()
```

[43]:

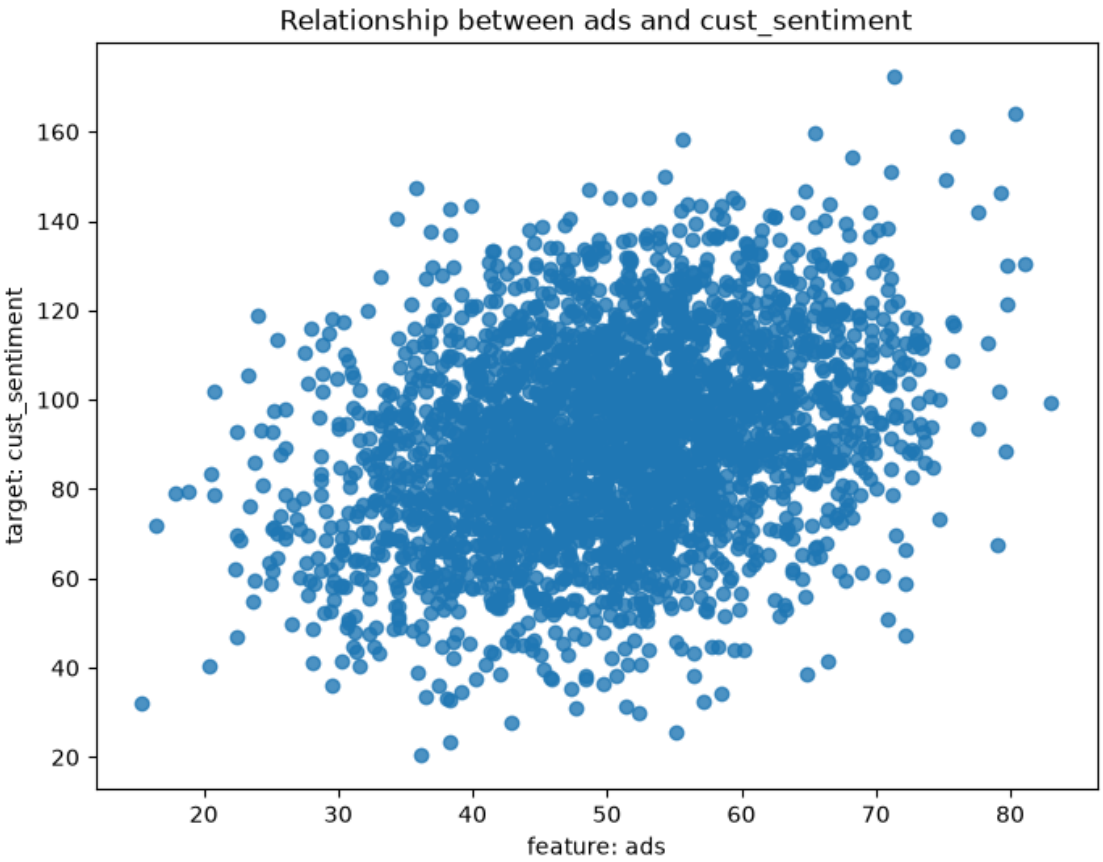
	cust_sentiment	ads	satis	contact	econ1	econ2
0	44.657012	57.764230	10.716213	31.352358	8.825752	515.602000
1	86.588110	60.547010	9.592415	27.784756	8.638531	470.754586
2	72.956784	46.305818	11.400391	27.825407	8.041998	564.201090
3	127.721010	33.096335	9.710857	28.832871	12.678198	490.286382
4	72.738182	38.392287	7.370844	28.498596	12.097519	421.748301

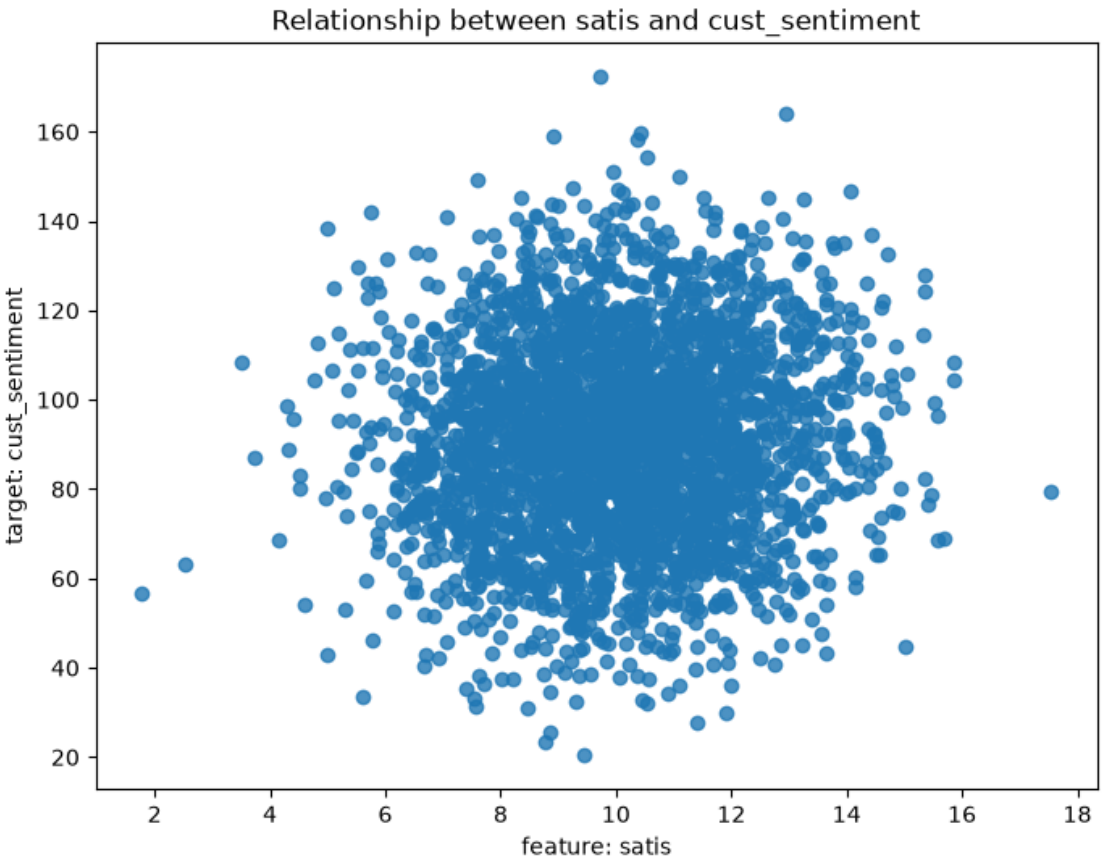
[52]:
```
def x_y_scatterplots(file_path: str, x_cols: list[str], y: str):

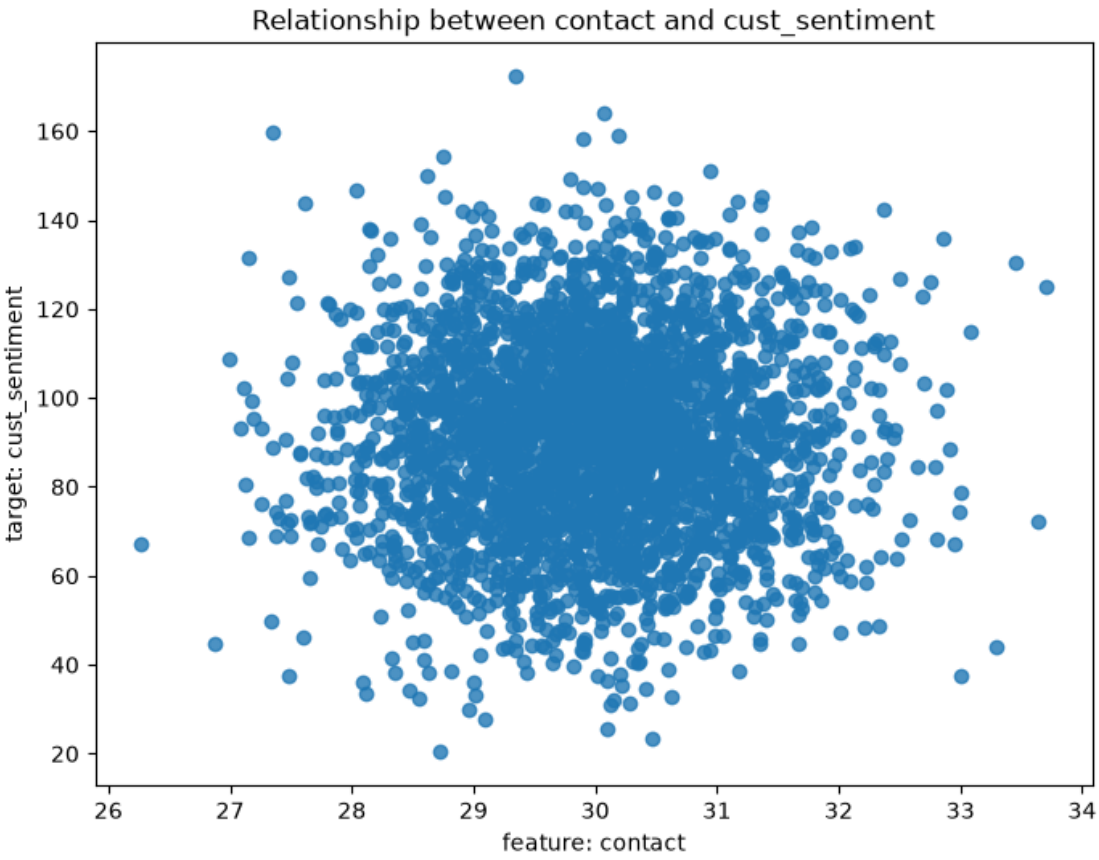
    data = pd.read_csv(file_path)

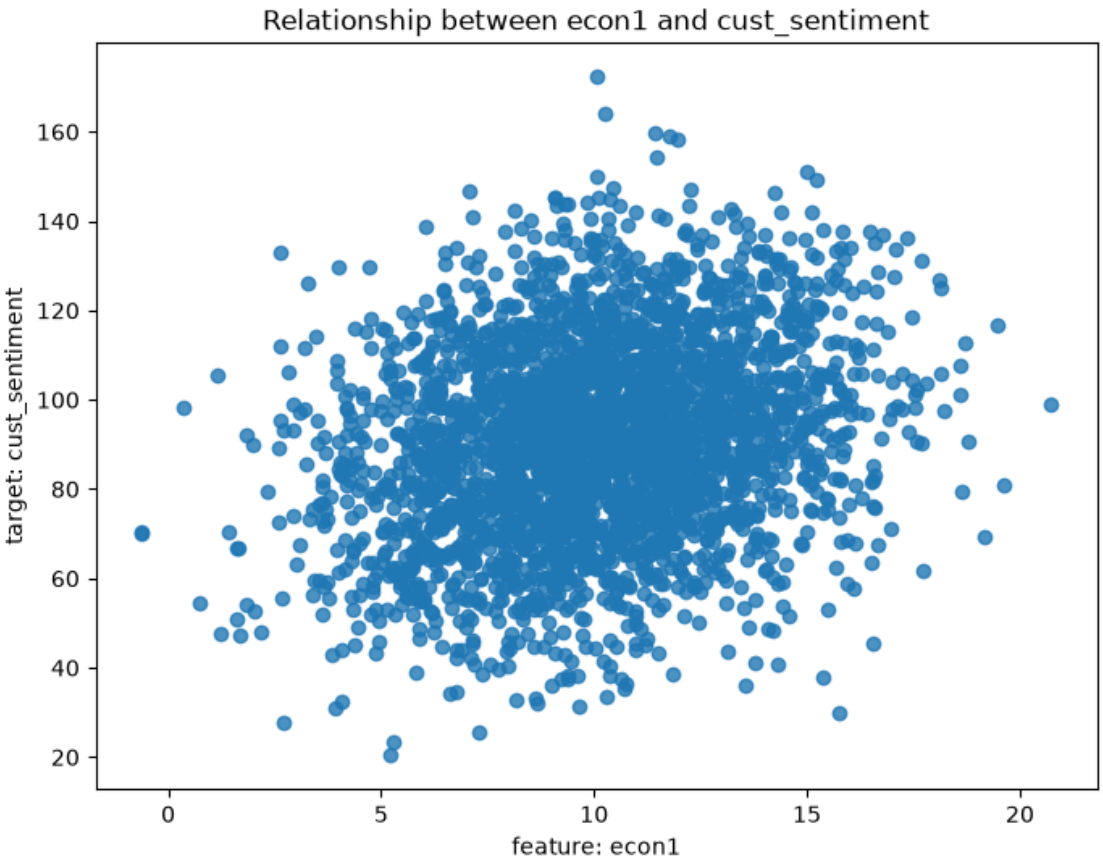
    for x in x_cols:
        plt.figure(figsize=(8, 6))
        plt.scatter(x = data[x], y = data[y], alpha = 0.8)
        plt.title(f"Relationship between {x} and {y}")
        plt.ylabel(f"target: {y}")
        plt.xlabel(f"feature: {x}")
        plt.show()
```

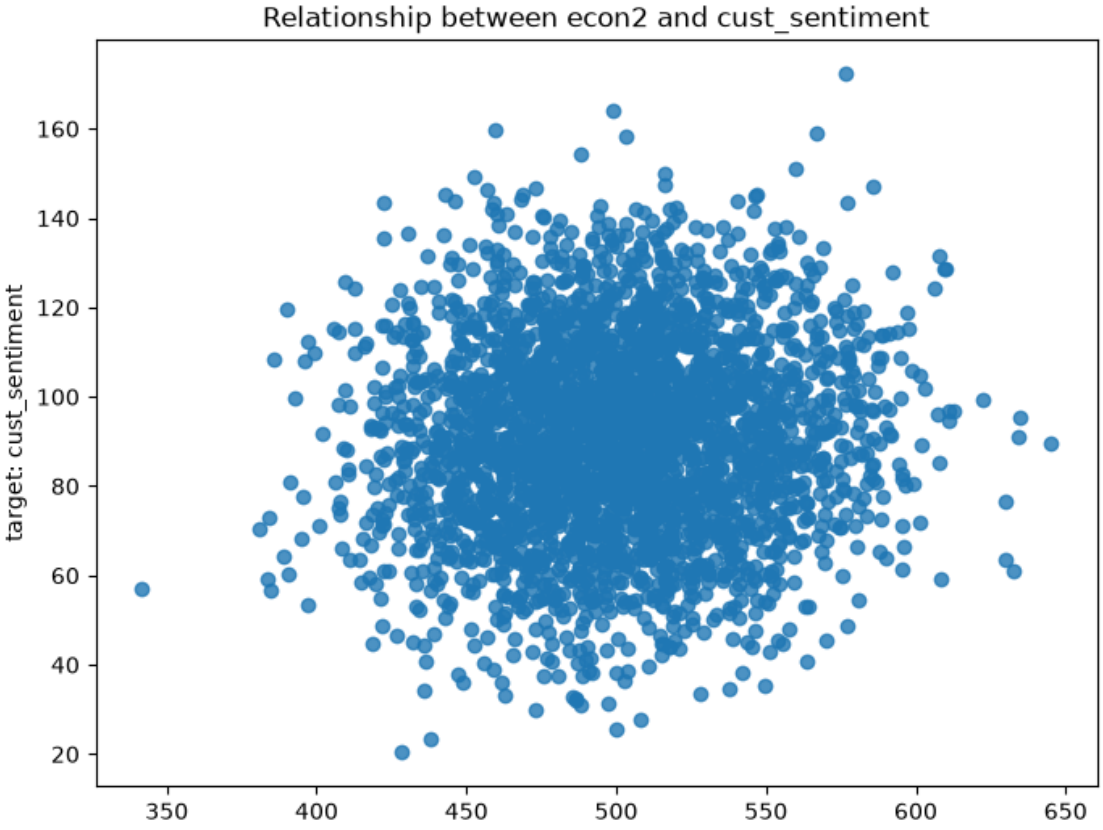
[53]:
```
x_cols = ["ads", "satis", "contact", "econ1", "econ2"]
x_y_scatterplots(file_path = 'reg1.csv', x_cols = x_cols, y = "cust_sentiment")
```


Relationship between ads and cust_sentiment
target: cust_sentiment
feature: ads
160
140
120
100
80
60
40
20
20
30
40
50
60
70
80


Relationship between satis and cust_sentiment
target: cust_sentiment
feature: satis
160
140
120
100
80
60
40
20
2
4
6
8
10
12
14
16
18


Relationship between contact and cust_sentiment
target: cust_sentiment
feature: contact
160
140
120
100
80
60
40
20
26
27
28
29
30
31
32
33
34


Relationship between econ1 and cust_sentiment
target: cust_sentiment
feature: econ1
160
140
120
100
80
60
40
20
0
5
10
15
20


Relationship between econ2 and cust_sentiment
target: cust_sentiment
160
140
120
100
80
60
40
20
350
400
450
500
550
600
650

Question 3:

Claude response part a)

Pick up an entry-level pair of binoculars

- Aim for an **8x42** configuration. The 8x magnification provides a steady hand-held image and a wide field of view, while the 42mm objective lenses gather ample light in dim forest cover or at dawn.

Download free identification apps

- Install **Merlin Bird ID** (by the Cornell Lab of Ornithology) for instant sound ID and step-by-step visual identification. Pair it with **eBird** to log your sightings, track life lists, and see recent local hotspots.

Master the four core identification markers

- Before reaching for a field guide, note four distinct features:
    - **Size & Shape:** Compare against familiar baselines (sparrow, robin, crow). Is the bill short and conical (seed-eater) or sharp and slender (insect-eater)?
    - **Color Pattern:** Look for distinct field marks like wing bars, eye stripes, or rump patches.
    - **Behavior:** Is it foraging on the ground, clinging vertically to tree bark, or catching insects mid-air?
    - **Habitat:** Note whether you are seeing it in dense forest understory, open marsh, or suburban brush.

Start in your immediate neighborhood

- Focus first on your backyard, balcony, or nearest community park. Getting intimately familiar with 10 to 15 resident species (such as cardinals, chickadees, starlings, or song sparrows) builds your baseline, making migratory or uncommon species stand out immediately.

Listen before you look

- Birds announce themselves well before you spot them visually. Step outside in the early morning during the "dawn chorus," pause in place, and track rustling leaves or distinct calls. Following sound cues with your naked eyes first—then raising binoculars directly to that line of sight—is the fastest way to spot birds in dense foliage.

Claude response part b)

Pick up an entry-level pair of binoculars

- Aim for an **8x42** configuration. The 8x magnification provides a steady hand-held image and a wide field of view to locate moving subjects, while the 42mm objective lenses gather ample light in dim forest canopy or at dawn.

Download free identification apps

- Install **Merlin Bird ID** (by the Cornell Lab of Ornithology) for real-time sound identification and step-by-step visual matching. Pair it with **eBird** to log your sightings, track your personal life list, and find active birding hotspots nearby.

Learn the four core identification markers

- Before looking up a bird, make note of four physical and environmental cues:
    - **Size & Silhouette:** Compare it against familiar baselines (sparrow, robin, crow). Check bill shape—short and stout indicates a seed-eater, while slender and pointed indicates an insect-eater.
    - **Field Marks:** Look for distinct coloration, eye stripes, wing bars, or tail patterns.
    - **Behavior:** Observe whether it feeds on open ground, climbs tree bark vertically, or hover-gleans foliage.
    - **Habitat:** Factor in the environment—reed beds, dense woodlots, open fields, or suburban gardens attract very different species.

Start in your immediate neighborhood

- Begin in your backyard, local park, or neighborhood walking path. Learning 10 to 15 resident species thoroughly (such as chickadees, northern cardinals, blue jays, or song sparrows) establishes your baseline so migrating or uncommon species stand out immediately.

Listen before you look

- Most birds make noise long before they show themselves. Head out during the early morning "dawn chorus," stop walking, and pinpoint rustling brush or distinctive calls with your naked ears and eyes first. Once you spot movement, raise your binoculars straight to your line of sight rather than hunting through the lenses.

ChatGPT response part a)

5 simple steps to start birdwatching

1. Start in your backyard or a nearby park

You don't need to travel far. Look for birds around trees, bushes, ponds, or bird feeders. Even a neighborhood walk is a great starting point.

2. Bring binoculars, but keep it simple

An 8×42 pair is a good beginner option, but you can start without binoculars. Your eyes and ears are enough to get started.

3. Learn to identify common birds

Start with five familiar species, such as robins, cardinals, blue jays, chickadees, and mourning doves. Notice their colors, sizes, shapes, and calls.

4. Use a bird identification app

Try Merlin Bird ID for identifying birds by appearance or sound. Keep a simple record of which species you observe and where you find them.

5. Practice for 10–15 minutes regularly

Early morning is often ideal because many birds are active and singing. Stay quiet, move slowly, and observe from a distance without disturbing them.

Your first birdwatching goal: Find and identify three different bird species during a 15-minute walk. You don't need expensive equipment or prior knowledge.

For free beginner resources, visit Merlin Bird ID or Cornell Lab's All About Birds

ChatGPT response part b:

1. Choose a location: Start in your backyard, a local park, or a nearby nature trail where birds are commonly found.

2. Gather basic equipment: Bring binoculars if available, a notebook, and a bird identification guide.

3. Learn common bird species: Familiarize yourself with local birds by studying their colors, sizes, shapes, and sounds.

4. Observe quietly: Find a comfortable spot, remain still, and watch birds without disturbing their natural behavior.

5. Record your observations: Write down the birds you see, where you found them, and any interesting behaviors. Practice regularly to improve your identification skills.

For the google source I used the website: https://www.audubon.org/birding/how-to-start-birding

The main differences I noticed between the google source and the AI is the level of depth for some of the responses. This is likely due to including 'simply' within the model prompt. The google source touches on practically all 5 points the LLMs make, just with slightly greater depth. For example, the google source mentions the importance of equipment, but mentions a larger set of items that helps with bird watching such as a weather-proof notebook. The google source also emphasizes prioritizing safety of you, the bird, and the environment whereas the LLMs do not emphasize the possible safety concerns.

As for starting for the very first time, the LLMs responses were close to identical to the google source in terms of first finding an area close to you, practicing observational skills, and familiarizing yourself with bird species characteristics.

d)

As for accuracy and consistency, the claude's responses were the most consistent and accurate. The consistency was very high as the response after telling the model to ignore previous instructions had almost no effect on the response. In fact, all of the 5 point bullet titles were the exact same, just some of the supporting body text was slightly different. Its points were the most accurate compared to the google search, with one exception being recommending a bird identification app which is not mentioned within the google source. ChatGPT had the same mention of an identification app as well.

ChatGPT responses were far less consistent. Ignoring previous instructions leads to a broader, shortened suggestion list as well as different suggestions. One example would be using an app for bird identification that wasn't listed anymore within the second recorded response. The equipment recommendation changes from not necessarily needing binoculars to

start bird watching within the first response compared to gathering equipment all the way from binoculars, writing equipment and having a bird identification guide.

Microsoft



Dennis Ousmanov

has successfully completed

Train and evaluate classification models

October 25, 2026

Satya Narayana Nadella

Microsoft



Dennis Ousmanov

has successfully completed

Train and evaluate regression models

October 25, 2026

Satya Narayana Nadella

feature: econ2

0.3 Question 5

Making the mistake of predicting no heart attack when there in fact is one (False Negative) is much worse than the other way arround. The cost with this error is much higher than the oppositte, predicting heart attack when there isnt one (False Positive). For a FP we can assign treatment to the patient, monitor them and provide overall medical assisatnce, however a FN can result in death or more expensive procedures such as intensive surgeries, ect.

A false negative in classification terms it means we predict the negative class when in fact the true label is positive. For this problem within the classification task, our goal is to then minimize false negatives, which can be done by assessing the Recall metric (TP / TP + FP). Ideally we want this to be as close as possible to 1, which means that the FP count is 0. The way I will implement this within the model is by penalizing mistakes for the positive class more than classification mistakes for the negative class.

[157]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix, recall_score, precision_score, f1_score,␣
 ↪precision_recall_curve, roc_curve, roc_auc_score
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
```

[158]:
```
heart = pd.read_csv("Heart.csv")
heart = heart.drop(columns = ['Unnamed: 0'])
heart.head()
```

[158]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	Thal	AHD
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	fixed	No
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	normal	Yes
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	reversable	Yes
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	normal	No
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	normal	No

[159]:
```
heart["target"] = np.where(heart["AHD"] == 'Yes', 1, 0)

heart = heart.drop(columns = ['AHD'])
```

[160]:
```
heart.columns
```

[160]:
```
Index(['Age', 'Sex', 'ChestPain', 'RestBP', 'Chol', 'Fbs', 'RestECG', 'MaxHR',
       'ExAng', 'Oldpeak', 'Slope', 'Ca', 'Thal', 'target'],
      dtype='str')
```

[161]:
```
# Training a LR classifier with default parameters and no penalization on␣
 ↪getting positive class predictions incorrect

lr = LogisticRegression(random_state=51, max_iter=1000)
preprocessor = ColumnTransformer(
    transformers = [
        ('categorical', OneHotEncoder(sparse_output= False, handle_unknown=␣
 ↪"ignore"), ['Thal', 'ChestPain']),
        ('num', StandardScaler(), ['Age', 'RestBP', 'Chol', 'MaxHR', 'Oldpeak'])
        ],
    remainder = 'passthrough'
)

X_train, X_test, y_train, y_test = train_test_split(heart.drop(columns =␣
 ↪'target'), heart["target"], test_size= 0.2, random_state= 51)
```

```
preprocessor.set_output(transform= "pandas")

X_train_ohe = preprocessor.fit_transform(X_train)

X_test_ohe = preprocessor.transform(X_test)
```

[162]:
```
print((X_train_ohe['categorical__Thal_nan'] == 1).mean())
print((X_train_ohe['categorical__Thal_nan'] == 1).sum())
print(X_train_ohe.isna().any(axis=1).sum())

print((X_test_ohe['categorical__Thal_nan'] == 1).mean())
print(X_test_ohe.isna().any(axis=1).sum())
```

```
0.008264462809917356
2
4
0.0
0
```

Since there are only 6 rows within the training set with missing values, I will drop those rows before fitting the LR classifier since they are not allowed.

[163]:
```
valid_rows = (X_train_ohe['categorical__Thal_nan'] != 1) & (~X_train_ohe.isna().
  ↪any(axis=1))
X_train_ohe = X_train_ohe[valid_rows]
y_train = y_train[valid_rows]
```

[168]:
```
lr.fit(X_train_ohe, y_train)

y_hat = lr.predict(X_test_ohe)

accuracy = accuracy_score(y_test, y_hat)
recall = recall_score(y_test, y_hat)
precision = precision_score(y_test, y_hat)
au_roc = roc_auc_score(y_test, y_hat)
auc_pr, auc_r, _ = precision_recall_curve(y_test, y_hat)
print(f"Accuracy: {accuracy:.4f}")
print(f"Recall: {recall:.4f}")
print(f"Precision: {precision:.4f}")
print(f"AU-ROC: {au_roc:.4f}")
```

```
Accuracy: 0.8525
Recall: 0.9200
Precision: 0.7667
AU-ROC: 0.8628
```

[170]:
```
confusion_matrix(y_test, y_hat)
```

[170]:
```
array([[29,  7],
       [ 2, 23]])
```

From the confusion matrix we have 2 false negatives. In the context of the heart problem that means we predict for 2 patients that they will not have a heart attack when they in fact do. Ideally wed like to get that to 0 which would maximize recall. Here I will to assign a larger penalty to errors for the positive class within the loss function to see if the number of FNs drop from 2 and are able to minimize the more costly mistake.

[177]:
```
lr2 = LogisticRegression(random_state=51, max_iter=1000, class_weight={0: 1, 1:␣
↪5})

lr2.fit(X_train_ohe, y_train)

y_hat = lr2.predict(X_test_ohe)

accuracy = accuracy_score(y_test, y_hat)
recall = recall_score(y_test, y_hat)
precision = precision_score(y_test, y_hat)
au_roc = roc_auc_score(y_test, y_hat)
auc_pr, auc_r, _ = precision_recall_curve(y_test, y_hat)
print(f"Accuracy: {accuracy:.4f}")
print(f"Recall: {recall:.4f}")
print(f"Precision: {precision:.4f}")
print(f"AU-ROC: {au_roc:.4f}")
```

```
Accuracy: 0.8033
Recall: 1.0000
Precision: 0.6757
AU-ROC: 0.8333
```

[178]:
```
confusion_matrix(y_test, y_hat)
```

[178]:
```
array([[24, 12],
       [ 0, 25]])
```

From the output of the classification matrix, we can see that we have no incorrect predictions for the positive class, ie we have minimized the most costly mistakes. Ideally we want to reduce the number of false postives as well since the since there are quite a several incorrect classification predictions for the true negative class labels. Obviously my model is not at a final production level, but enough to illustrate minimizing the most costly error based on the domain.